

CS1CA Computer Systems Architecture

Processor Pipelining (4)

Week 9 lecture 2 of autumn semester 2025

Lectured by

Dr. Hong Wei

h.wei@reading.ac.uk

Room 133 in Polly Vacher

Instr No.	Pipeline Stage						
1	IF	ID	EX	MEM	WB		
2		IF	ID	EX	MEM	WB	
3			IF	ID	EX	MEM	WB
4				IF	ID	EX	MEM
5					IF	ID	EX
Clock Cycle	1	2	3	4	5	6	7

In this lecture

- The following contents will be delivered
 - Continue the discussion on control hazards
 - Performance with pipeline stall cycles from branch
 - DLX pipeline to handle multi-cycle (floating point) operations
 - Summary: the MIPS R4000 pipeline (self-learning)
- The learning outcomes are listed below
 - To be able to calculate the performance of pipelined machine affected by branch stall cycles.
 - To be aware of pipeline issues in dealing with floating point operations

Performance with pipeline stall cycles from branch

$$\text{pipeline Speedup} = \frac{\text{Pipeline depth}}{1 + \text{Pipeline stall cycles from branch}}$$

Pipeline stall cycles from branch

Therefore

$$\text{pipeline Speedup} = \frac{\text{Pipeline depth}}{1 + \text{branch frequency} \times \text{branch penalty}}$$

Note: To increase the pipeline speedup, we need to reduce “branch penalty”, i.e. reduce “branch stall cycles”.

Example:

A benchmark program is running in a 5-stage pipelined DLX machine. It is known that branch instructions take 10% of all instructions (branch frequency = 10%), and branch penalty is 1 clock cycle. Calculate the performance gain from pipelining only taking branch stall cycles into account.

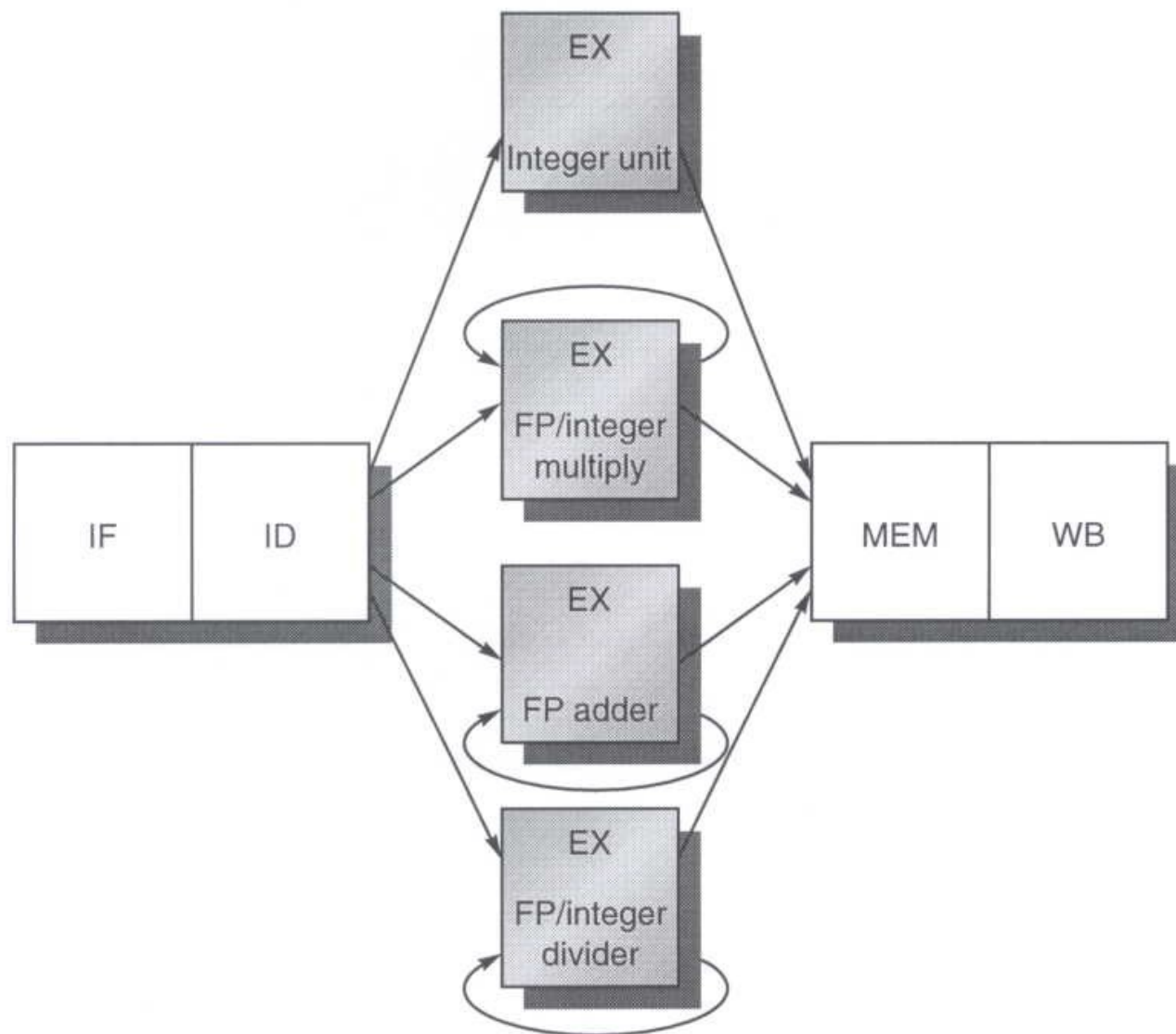
$$\text{Pipeline Speedup} = \frac{\text{Pipeline depth}}{1 + \text{branch frequency} \times \text{branch penalty}}$$
$$= 4.55$$

DLX pipeline to handle multicycle operations

*To handle floating-point operations in DLX pipeline, **separate functional units** are needed.*

- Integer unit handling loads, stores, integer ALU operations and branches; (original unit)
- FP and integer multiplication; (additional unit)
- FP adder handling FP addition, subtraction, and conversion; (additional unit)
- FP and integer division (additional unit)

The DLX pipeline with three additional functional units



Latencies and initiation intervals for functional units

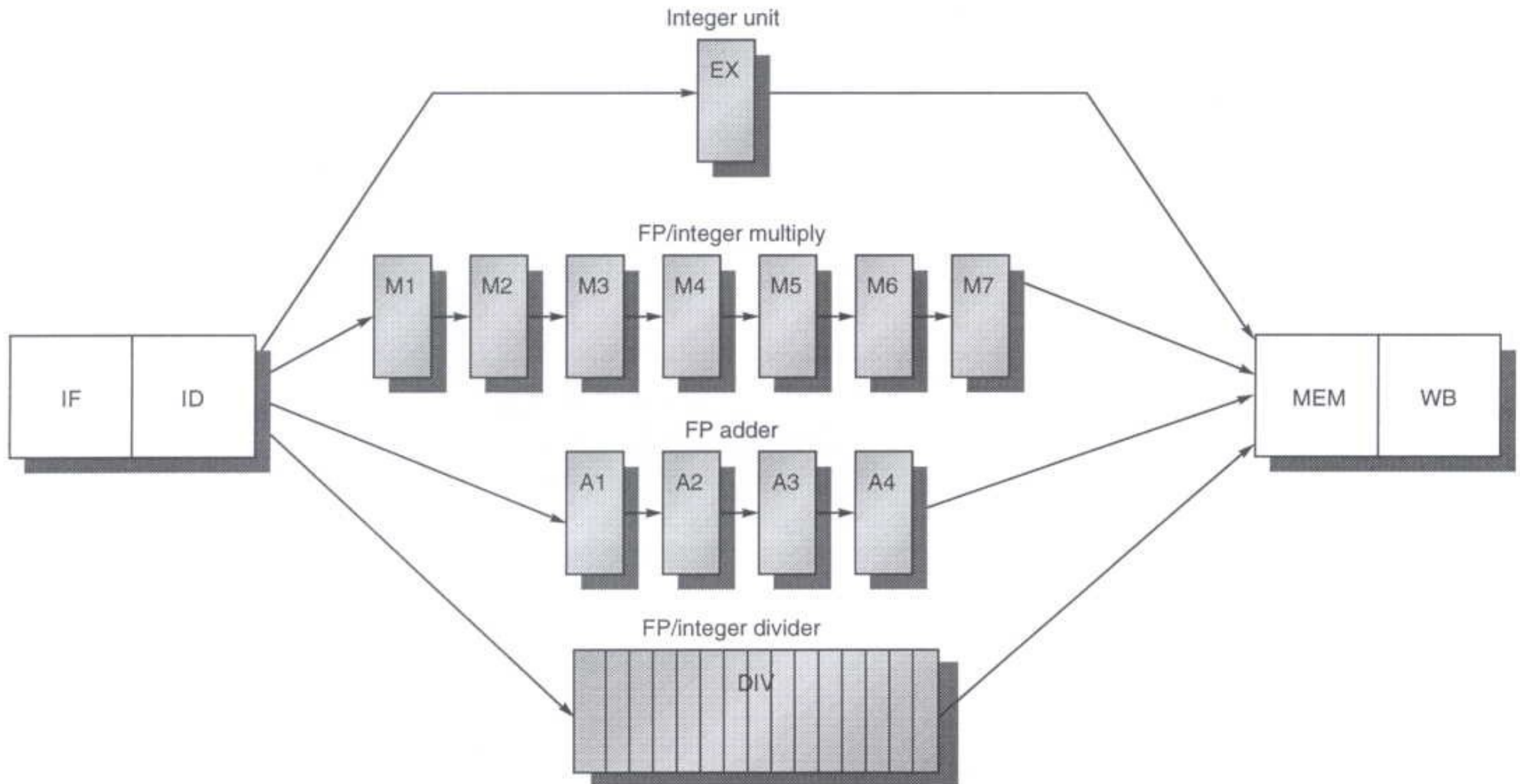
Functional Unit	Latency (cc)	Initiation interval (cc)
Integer ALU	0	1
Data memory (integer and FP loads)	1	1
FP add	3	1
FP and integer multiply	6	1
FP and integer divide	24	25

cc: clock cycle

Latency: the number of clock cycles between an instruction to start its execution stage and to produce the result.

Initiation interval: clock cycles that must elapse between issuing two operations.

A pipeline that supports FP operations in DLX



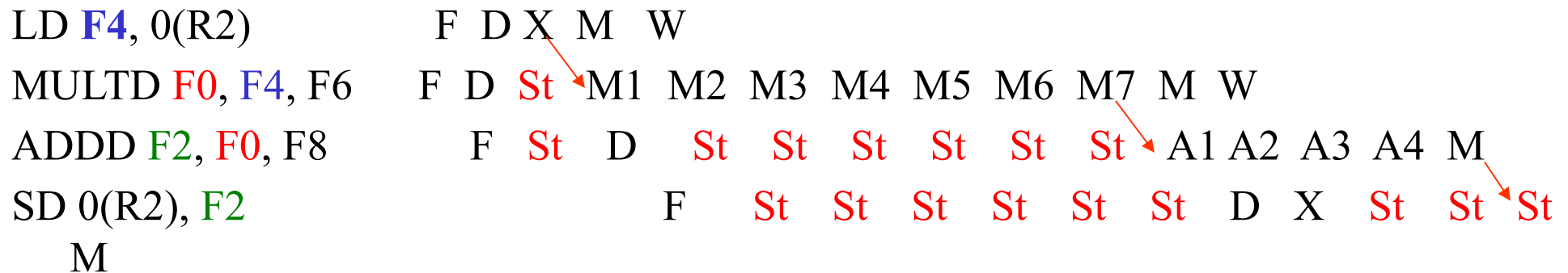
Problems in FP DLX pipelining

- Structure hazards, e.g. divider unit which is not fully pipelined.
- Number of register writes required in a cycle can be larger than 1 (simultaneously write-back).
- WAW hazards occur as instructions no longer reach the WB stage in order.
- RAW hazards exist as those in the integer pipeline.
- Exceptions: when occur, how to deal with instructions that may complete in different order from that they were issued (Think about these scenarios, but will not be discussed in this module)

Example 1

IF:F; ID:D; Ex:X; MEM:M; WB:W; Stall:St;
Addition:An; Multiplication:Mn, where $n=1,2,\dots,7$

- RAW hazards: this FP code sequence shows the stalls arising from RAW hazards with forwarding applied.



Example 2

IF:F; ID:D; Ex:X; MEM:M; WB:W; Stall:St;
Adding:An; Multiplication:Mn, where $n=1,2,\dots,7$

- WAW hazards: a typical FP code sequence showing possible WAW hazards

MULTD	F	D	M1	M2	M3	M4	M5	M6	M7	M	W
ADDD	F	D	A1	A2	A3	A4	M	W			
LD			F	D	E	M	W				
SD					F	D	E	M	W		
ADD					F	D	E	M	W		

Write-back in
different order
from issuing
instructions

Example 3

- Simultaneous write-back (structure hazard): three instructions try to perform a write back to the FP register file simultaneously.

	cc1											cc11									
MULTD F0, F4, F6	F	D	M1	M2	M3	M4	M5	M6	M7	M	W										
.....			F	D	X	M	W														
.....			F	D	X	M	W														
ADDD F2, F4, F6	F	D	A1	A2	A3	A4	M	W													
.....				F	D	X	M	W													
.....					F	D	X	M	W												
LD F2, 0(R2)						F	D	X	M	W											

Instruction set design and pipelining

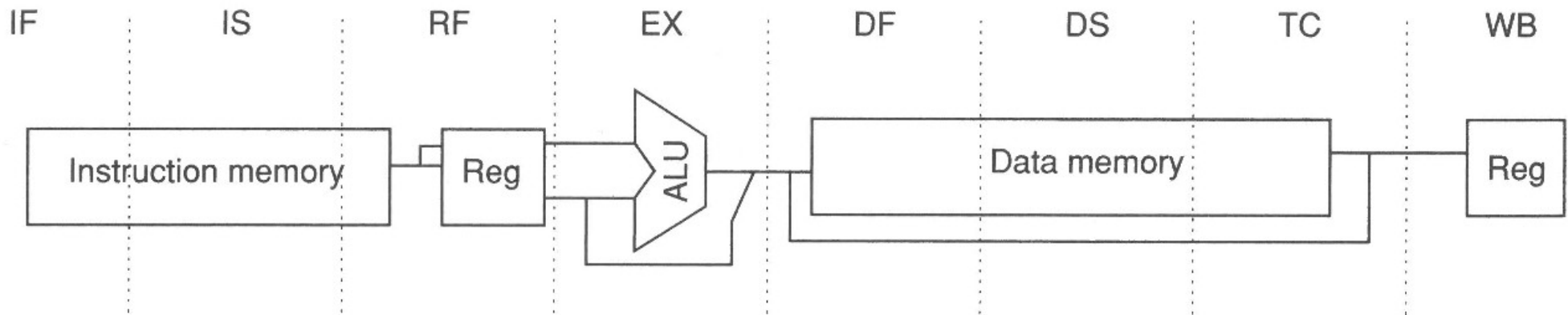
- Variable instruction lengths and running times can lead to imbalance among pipeline stages;
- Sophisticated addressing modes can lead to different sorts of problem;
- Architectures that allow writes into the instruction space can cause trouble for pipelining;
- Implicitly setting condition codes increase the difficulty of finding when a branch has been decided and the difficulty of scheduling branch delays.

Putting it all together: MIPS R4000 pipeline

- Please refer to Hennessy & Patterson's book (3rd edition), ppA57-A66 to review pipelining.
 - Super-pipelining: eight pipeline stages;
 - Increased memory access clock cycles;
 - Forwarding technique;
 - Predict-untaken strategy for the branch delay;
 - Floating-point pipeline: three functional units – a floating-point divider, a floating-point multiplier, and a floating-point adder.
 - Performance of the R4000 pipeline.

Putting it all together: MIPS R4000 pipeline

- The 8 pipeline stages are shown below.



IF (two clock cycles): first half of instruction fetch; IS: second half of instruction fetch.

RF: instruction decode, register fetch, and hazard checking.

EX: execution stage – calculation of results including effective memory address.

DF (three clock cycles): first half of data fetch; DS: second half of data fetch; TC: tag comparison.

WB: write back for loads and register-register operation.

Summary of the lecture

- Control hazards
 - Performance issue when branch stall cycles are needed.
- DLX pipeline to deal with floating point operations
 - Additional units are added to the original pipeline
 - Structure hazards, data hazards (possible WAW, WAR, and RAW), and control hazards in the pipeline datapath when dealing with floating point operations.
 - All techniques introduced in the original pipeline can be used to deal with pipeline hazards in the floating point operations.

End of this lecture: questions and exercises

- A benchmark program is running in an 8-stage pipelined MIPS machine. It is known that branch instructions take 15% of all instructions (branch frequency = 15%), and branch penalty is 2 clock cycle. Calculate the performance gain from pipelining only taking branch stall cycles into account.
- When pipelining is implemented to handle floating point operations, what special issues are concerned?
- Using an example, explain WAW data hazards in pipelined floating point operations.